

EchoMind 完整使用指南

前言

有需要的可以下载pdf的文档。但是请不要倒卖，这个项目我沉淀了很久，定价也不是很高，感谢🙏
(最后有用的话记得多推荐一下😊)

求反馈（重要

大家觉得这个项目怎么样？用过这个项目的或者看完项目的可以在小红书找我给我反馈一下用着怎么样（球球了，想收集一波建议，也找一下优化方向）

本文档说明 EchoMind 的部署、启动、API 调用、知识库使用、ChromaDB 数据查看、监控评测和常见排障。

EchoMind 是一个企业级智能客服系统，核心链路为：

代码块

```
1  用户请求
2      -> FastAPI /chat
3      -> MemoryManager 读取 Redis 工作记忆 + ChromaDB 情景记忆 + 用户画像
4      -> IntentRecognizer 识别意图
5      -> AgentOrchestrator 路由到 General/Technical/Billing Agent
6      -> LLM 生成回复
7      -> 写入 Redis, 并异步更新 ChromaDB 用户画像
```

1. 项目结构

代码块

```

1  EchoMind/
2  |—— api/main.py          # FastAPI 入口, /chat /search /knowledge
   /monitor /eval
3  |—— core/intent_recognizer.py  # 三路融合意图识别
4  |—— agents/agent_orchestrator.py # 多 Agent 路由编排
5  |—— memory/conversation_memory.py # Redis + ChromaDB 记忆管理
6  |—— mcp/tool_manager.py      # MCP 工具调用、查询改写、重排、熔断、缓存、降级
7  |—— mcp/knowledge_base.py    # ChromaDB RAG 知识库
8  |—— monitor/performance_monitor.py # Agent/工具在线监控

```

```
9 |— evaluation/evaluator.py      # 端到端评测
10 |— data/demo_docs/            # 演示知识库文档
11 |— docker-compose.yml         # Docker 全栈编排
12 |— Dockerfile
13 |— requirements.txt
14 |— .env
```

2. 环境准备

2.1 必需依赖

- Docker
- Docker Compose
- Anthropic API Key，或兼容 Anthropic 协议的第三方 API Key

2.2 配置 `.env`

复制示例文件：

代码块

```
1 cp .env.example .env
```

最少需要配置：

代码块

```
1 ANTHROPIC_API_KEY=your_api_key
```

如果使用 DeepSeek 这类 Anthropic 兼容接口，可以配置：

代码块

```
1 ANTHROPIC_BASE_URL=https://api.deepseek.com/anthropic
2 ANTHROPIC_MODEL=deepseek-chat
3 ANTHROPIC_API_KEY=your_deepseek_key
```

Docker Compose 场景下，Redis 和 ChromaDB 的连接由 `docker-compose.yml` 覆盖为容器内地址。通常不需要手动改：

代码块

```
1 REDIS_PASSWORD=echomind123
2 CHROMA_HOST=localhost
3 CHROMA_PORT=8001
```

2.3 全栈部署和 run 开发模式的区别

EchoMind 常用两种 Docker 启动方式：`docker compose up` 全栈部署，以及 `docker run` 开发模式。两者最大的区别是：**全栈部署会同时启动应用和依赖服务**；**run 开发模式通常只手动运行一个应用容器，依赖服务需要提前启动。**

选择建议：

- 想完整体验 HTTP API、Swagger、Nginx、Prometheus：用 **Docker Compose 全栈部署**。
- 想调试源码或 CLI，并且希望本地改代码后快速重跑：用 **Docker run 开发模式**。
- 如果只是跑 CLI，最省心的方式是 `docker compose run --rm echomind python api/main.py --cli`，它会自动使用 Compose 网络。

3. Docker Compose 全栈部署

推荐使用此方式启动完整服务。

代码块

```
1 docker compose up -d --build
```

查看服务状态：

代码块

```
1 docker compose ps
```

查看应用日志：

代码块

```
1 docker compose logs -f echomind
```

看到 EchoMind 启动日志并且健康检查通过后，服务可用。

启动后的端口：

服务	容器名	宿主机端口	容器内端口	用途
EchoMind API	echomind-app	8000	8000	主 API 服务
Nginx	echomind-nginx	80	80	反向代理
ChromaDB	echomind-chromadb	8001	8000	向量数据库
Redis	echomind-redis	6379	6379	工作记忆
Prometheus	echomind-prometheus	9090	9090	监控数据
latency_ms	端到端耗时	挂载 -v "\$(pwd):/workspace" 后， 代码修改可直接生效，重启容器即可		
results	每条评测结果	本地开发、调试 CLI、临时覆盖环境变量		
常见问题	API Key 或依赖健康检查失败	忘记启动 Redis/ChromaDB，导致 redis:6379 Name or service not known		

健康检查：

代码块

```
1 curl http://localhost:8000/health
```

Swagger 文档：

代码块

```
1 http://localhost:8000/docs
```

也可以通过 Nginx 访问：

代码块

```
1 curl http://localhost/health
```

4. Docker Run 开发模式

开发时可以只用 Compose 启动依赖，然后用 `docker run` 挂载当前代码目录。

先启动 Redis 和 ChromaDB：

代码块

```
1 docker compose up -d redis chromadb
```

构建镜像：

代码块

```
1 docker compose build echomind
```

启动 HTTP 服务：

代码块

```
1 docker run -it --rm \  
2   --network echomind_echomind-network \  
3   -p 8000:8000 \  
4   -e ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic" \  
5   -e ANTHROPIC_API_KEY="your_key" \  
6   -e ANTHROPIC_MODEL="deepseek-chat" \  
7   -e REDIS_URL="redis://:echomind123@redis:6379/0" \  
8   -e CHROMA_HOST="chromadb" \  
9   -e CHROMA_PORT="8000" \  
10  -e CHROMA_PERSIST_DIRECTORY="/workspace/data/chroma" \  
11  -v "$(pwd):/workspace" \  
12  -w /workspace \  
13  echomind
```

CLI 交互模式：

代码块

```
1  docker run -it --rm \
2    --network echomind_echomind-network \
3    -e ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic" \
4    -e ANTHROPIC_API_KEY="your_key" \
5    -e ANTHROPIC_MODEL="deepseek-chat" \
6    -e REDIS_URL="redis://:echomind123@redis:6379/0" \
7    -e CHROMA_HOST="chromadb" \
8    -e CHROMA_PORT="8000" \
9    -v "$(pwd):/workspace" \
10   -w /workspace \
11   echomind \
12   python api/main.py --cli
```

5. Swagger 和接口总览

EchoMind 基于 FastAPI 构建，启动 HTTP 服务后可以直接在浏览器访问 Swagger UI 调用接口。

本地 Swagger 地址：

代码块

```
1  http://localhost:8000/docs
```

如果使用 Nginx 反向代理：

代码块

```
1  http://localhost/docs
```

打开 Swagger 后，可以点击任意接口右侧的 **Try it out**，填写参数后点 **Execute** 直接调用本地服务。

常用调试顺序：

代码块

1	1. GET /health	确认服务是否就绪
2	2. POST /chat	测试主对话链路
3	3. GET /knowledge/stats	查看知识库是否已有数据
4	4. POST /knowledge/upload	上传演示知识库文件
5	5. POST /search	测试知识库检索、查询改写和重排
6	6. GET /monitor	查看 Agent 和工具运行指标
7	7. GET /skills	查看已加载 Skills
8	8. POST /skills/reload	重新加载 Skills
9	9. POST /eval/run	运行端到端评测

5.1 接口总览

方法	路径	参数位置	作用	适合场景
GET	/health	无	健康检查，返回服务状态和 Agent 统计	启动后确认服务可用
POST	/chat	JSON Body	主对话接口，完成记忆读取、意图识别、Agent 路由、回复生成、记忆写入	业务主链路
GET	/monitor	无	查看 Agent/工具统计、告警和优化建议	观察在线表现
POST	/search	Query 参数	执行知识库检索优化链路：查询改写、并行召回、合并去重、LLM 重排	测试 RAG 检索
GET	/skills	无	查看当前加载的 Skills、匹配关键词和解析错误	确认动态能力是否生效
POST	/skills/reload	无	运行时重新扫描 Skill 目录	修改业务规则后热加载
POST	/knowledge/add	JSON Body	批量导入文档到 ChromaDB 知识库	程序化导入文档
POST	/knowledge/upload	Form File	上传 .txt、.md、.json 文件导入知识库	手动上传知识库文件
GET	/knowledge/stats	无	查看知识库文档片段总数	确认知识库是否有数据
POST	/eval/run	无	运行内置意图识别和端到端对话评测	演示 LLM-as-Judge 评测
GET	/docs	浏览器访问	Swagger UI	浏览和调试所有接口

5.2 Skills 动态能力加载

EchoMind 支持从目录加载 Skills，用来把业务流程、客服话术、排障 SOP 等规则动态注入 Agent。
默认配置：

代码块

```
1 ECHOMIND_SKILLS_DIR=./skills
```

```
2 ECHOMIND_SKILLS_MAX_PROMPT_CHARS=5000
```

推荐结构：

代码块

```
1 skills/refund/SKILL.md
2 skills/customer_support/SKILL.md
```

`SKILL.md` 示例：

代码块

```
1 ---
2 name: 退款处理流程
3 description: 退款场景的客服处理规则
4 keywords: 退款,退费,refund
5 agents: billing,general
6 enabled: true
7 ---
8
9 # 退款处理流程
10
11 - 先确认订单号和支付方式。
12 - 涉及实际退款操作时转人工审核。
```

查看加载结果：

代码块

```
1 curl http://localhost:8000/skills
```

修改 Skill 文件后热加载：

代码块

```
1 curl -X POST http://localhost:8000/skills/reload
```

5.3 `/health`

用途：确认服务是否初始化完成。


```
1 curl http://localhost:8000/health
```

响应示例：

代码块

```
1  {
2    "status": "ok",
3    "agents": {
4      "general_0": {
5        "total": 0,
6        "success_rate": 1.0,
7        "avg_ms": 0.0,
8        "monitor_penalty": 0.0,
9        "routing_score": 1.0
10     }
11   }
12 }
```

5.4 /chat

用途：主对话接口。

请求体：

代码块

```
1  {
2    "message": "我要退款",
3    "user_id": "user_001",
4    "conv_id": "session_001"
5  }
```

字段说明：

返回字段：

5.5 /search

用途：测试 MCP 工具调用和 RAG 检索优化。

Query 参数：

示例：

代码块

```
1 curl -X POST "http://localhost:8000/search?query=退款多久到账&top_k=3"
```

5.6 /knowledge/add

用途：通过 JSON 批量导入知识库。

请求体：

代码块

```
1 {
2   "documents": [
3     {
4       "title": "退款政策",
5       "content": "用户在购买后 7 天内可以申请无理由退款..."
6     }
7   ]
8 }
```

5.7 /knowledge/upload

用途：上传文件导入知识库。

支持格式：

示例：

代码块

```
1 curl -X POST http://localhost:8000/knowledge/upload \  
2 -F "file=@data/demo_docs/sample_knowledge.json"
```

5.8 /knowledge/stats

用途：查看知识库片段数量。

代码块

```
1 curl http://localhost:8000/knowledge/stats
```

5.9 /monitor

用途：查看 Agent 和工具在线指标。

代码块

```
1 curl http://localhost:8000/monitor
```

返回内容包括：

5.10 /eval/run

用途：运行内置评测。

代码块

```
1 curl -X POST http://localhost:8000/eval/run
```

返回内容包括：

6. 使用项目

6.1 主对话接口

请求：

代码块

```
1 curl -X POST http://localhost:8000/chat \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "message": "我的订单什么时候到？",  
5     "user_id": "user_001",  
6     "conv_id": "session_001"  
7   }'
```

响应示例：

代码块

```
1  {  
2    "conv_id": "session_001",  
3    "response": "请提供订单号，我可以帮您查询订单状态和物流进度。",  
4    "intent": "query",  
5    "agent_type": "general",  
6    "escalated": false,
```

```
7     "latency_ms": 1234.5
8 }
```

字段说明：

6.2 多轮对话

多轮对话只需要保持同一个 `user_id` 和 `conv_id`。

代码块

```
1 curl -X POST http://localhost:8000/chat \
2     -H "Content-Type: application/json" \
3     -d '{
4         "message": "订单号是 A123456",
5         "user_id": "user_001",
6         "conv_id": "session_001"
7     }'
```

系统会从 Redis 读取当前会话最近消息，并从 ChromaDB 读取相关历史和用户画像，拼成上下文传给 Agent。

6.3 技术问题示例

代码块

```
1 curl -X POST http://localhost:8000/chat \
2     -H "Content-Type: application/json" \
3     -d '{
4         "message": "应用登录一直报 401 错误",
5         "user_id": "user_tech",
6         "conv_id": "tech_001"
7     }'
```

预期会路由到 `technical` Agent。

6.4 账单问题示例

代码块

```
1 curl -X POST http://localhost:8000/chat \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "message": "为什么这个月重复扣款了？我要退款",  
5     "user_id": "user_bill",  
6     "conv_id": "bill_001"  
7   }'
```

预期会路由到 `billing` Agent。

6.5 复合问题示例

代码块

```
1 curl -X POST http://localhost:8000/chat \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "message": "登录报错 401，而且这个月还重复扣款了",  
5     "user_id": "user_mix",  
6     "conv_id": "mix_001"  
7   }'
```

这类问题会触发多 Agent 并行协作，由技术 Agent 和账单 Agent 分别处理后合并回复。

7. 知识库使用

EchoMind 的知识库由 `mcp/knowledge_base.py` 管理，底层使用 ChromaDB collection：

代码块

```
1 knowledge_base
```

首次启动时，如果知识库为空，会自动导入默认客服文档，包括退款政策、订单查询、账户安全、技术故障排查、会员积分、配送说明。

7.1 查看知识库统计

代码块

```
1 curl http://localhost:8000/knowledge/stats
```

响应示例：

代码块

```
1 {
2   "total_chunks": 18
3 }
```

7.2 批量导入文档

代码块

```
1 curl -X POST http://localhost:8000/knowledge/add \
2   -H "Content-Type: application/json" \
3   -d '{
4     "documents": [
5       {
6         "title": "退换货政策",
7         "content": "用户在购买后 7 天内可以申请无理由退货，审核通过后 5-7 个工作日退
      款。"
8       },
9       {
10        "title": "会员权益",
11        "content": "金卡会员享受 9 折优惠，生日当月可获得双倍积分。"
12      }
13    ]
14  }'
```

系统会把长文档切成 500 字左右的片段，并写入 ChromaDB。

7.3 上传文件导入知识库

上传 Markdown：

代码块

```
1 curl -X POST http://localhost:8000/knowledge/upload \
2   -F "file=@data/demo_docs/troubleshooting.md"
```

上传 JSON：

```
1 curl -X POST http://localhost:8000/knowledge/upload \
2     -F "file=@data/demo_docs/sample_knowledge.json"
```

JSON 格式必须是数组：

代码块

```
1  [
2    {
3      "title": "文档标题",
4      "content": "文档内容"
5    }
6  ]
```

7.4 检索知识库

代码块

```
1 curl -X POST "http://localhost:8000/search?query=退款需要多久到账&top_k=3"
```

响应示例：

代码块

```
1  {
2    "query": "退款需要多久到账",
3    "results": [
4      {
5        "title": "退款政策",
6        "content": "审核通过后, 款项将在 5-7 个工作日内退回原支付账户。",
7        "score": 0.82,
8        "chunk": 0
9      }
10   ],
11   "reranked": true
12 }
```

`/search` 使用的是完整检索优化链路：

代码块

```
1  原始查询
2    -> LLM 查询改写成多个角度
3    -> 多个子查询并行召回 ChromaDB
```


- 4 -> 合并去重
- 5 -> LLM 重排
- 6 -> 返回 Top-K

8. ChromaDB 在项目中的用途

EchoMind 使用了三个 ChromaDB collection:

数据写入时机:

9. 在 Docker 中查看 ChromaDB 内容

Compose 中 ChromaDB 容器名是:

代码块

```
1 echomind-chromadb
```

宿主机访问端口是:

代码块

```
1 http://localhost:8001
```

容器内部端口是:

代码块

```
1 http://localhost:8000
```

9.1 查看 ChromaDB 是否存活

宿主机执行：

代码块

```
1 curl http://localhost:8001/api/v1/heartbeat
```

容器内执行：

代码块

```
1 docker exec -it echomind-chromadb curl http://localhost:8000/api/v1/heartbeat
```

9.2 查看所有 collection

代码块

```
1 curl http://localhost:8001/api/v1/collections
```

如果 ChromaDB 版本返回 tenant/database 相关错误，可以使用 Python 客户端查看，见下一节。

9.3 用 Python 客户端查看 collections

进入应用容器：

代码块

```
1 docker exec -it echomind-app bash
```

在容器里执行：

代码块

```
1 python - <<'PY'  
2 import chromadb  
3  
4 client = chromadb.HttpClient(host="chromadb", port=8000)  
5 print("heartbeat:", client.heartbeat())  
6  
7 collections = client.list_collections()  
8 print("collections:")  
9 for c in collections:  
10     print("-", c.name, "count=", c.count())
```

预期可以看到：

代码块

```
1 collections:
2   - knowledge_base count= ...
3   - episodic count= ...
4   - user_profile count= ...
```

9.4 查看 knowledge_base 文档内容

代码块

```
1 docker exec -it echomind-app bash
```

执行：

代码块

```
1 python - <<'PY'
2 import chromadb
3
4 client = chromadb.HttpClient(host="chromadb", port=8000)
5 col = client.get_collection("knowledge_base")
6
7 data = col.get(limit=10, include=["documents", "metadatas"])
8 for i, doc_id in enumerate(data["ids"]):
9     print("=" * 80)
10    print("id:", doc_id)
11    print("metadata:", data["metadatas"][i])
12    print("document:", data["documents"][i][:500])
13 PY
```

9.5 查询 knowledge_base

代码块

```
1 docker exec -it echomind-app bash
```

执行：

代码块

```
1 python - <<'PY'
2 import chromadb
3
4 client = chromadb.HttpClient(host="chromadb", port=8000)
5 col = client.get_collection("knowledge_base")
6
7 result = col.query(
8     query_texts=["退款多久到账"],
9     n_results=3,
10    include=["documents", "metadatas", "distances"],
11 )
12
13 for doc, meta, dist in zip(
14     result["documents"][0],
15     result["metadatas"][0],
16     result["distances"][0],
17 ):
18     print("=" * 80)
19     print("title:", meta.get("title"))
20     print("distance:", dist)
21     print("content:", doc[:300])
22 PY
```

9.6 查看用户画像 user_profile

先多调用几次 `/chat`，让系统异步生成用户画像：

代码块

```
1 curl -X POST http://localhost:8000/chat \
2     -H "Content-Type: application/json" \
3     -d '{"message": "我经常咨询会员积分和退款问题，回答请简洁一点", "user_id":
4         "profile_user", "conv_id": "profile_session"}'
```

等待几秒后查看：

代码块

```
1 docker exec -it echomind-app bash
```

代码块

```
1 python - <<'PY'
```

```

2  import json
3  import chromadb
4
5  client = chromadb.HttpClient(host="chromadb", port=8000)
6  col = client.get_collection("user_profile")
7
8  data = col.get(
9      where={"user_id": "profile_user"},
10     include=["documents", "metadatas"],
11 )
12
13 for i, doc in enumerate(data["documents"]):
14     print("=" * 80)
15     print("metadata:", data["metadatas"][i])
16     print(json.dumps(json.loads(doc), ensure_ascii=False, indent=2))
17 PY

```

9.7 查看情景记忆 **episodic**

情景记忆只有在当前会话消息数量达到压缩阈值后才会写入。默认阈值在

`MemoryManager.COMPRESS_AT` 中，目前是 15 条消息。

可以连续发送多条消息触发压缩：

代码块

```

1  for i in $(seq 1 16); do
2      curl -s -X POST http://localhost:8000/chat \
3          -H "Content-Type: application/json" \
4          -d '{"message": "这是第 $i 条测试消息，我想咨询退款和订单问题",
5              "user_id": "episodic_user", "conv_id": "episodic_session"}' > /dev/null
6  done

```

查看情景记忆：

代码块

```

1  docker exec -it echomind-app bash

```

代码块

```

1  python - <<'PY'
2  import chromadb
3
4  client = chromadb.HttpClient(host="chromadb", port=8000)

```

```

5 col = client.get_collection("episodic")
6
7 data = col.get(
8     where={"user_id": "episodic_user"},
9     include=["documents", "metadatas"],
10 )
11
12 for i, doc in enumerate(data["documents"]):
13     print("=" * 80)
14     print("metadata:", data["metadatas"][i])
15     print("summary:", doc)
16 PY

```

9.8 查看 ChromaDB 持久化文件

ChromaDB 的持久化卷在 Compose 中定义为：

代码块

```

1 volumes:
2     chromadb-data:

```

查看 Docker volume：

代码块

```

1 docker volume ls | grep chromadb
2 docker volume inspect echomind_chromadb-data

```

查看容器内数据目录：

代码块

```

1 docker exec -it echomind-chromadb sh
2 ls -lah /chroma/chroma
3 find /chroma/chroma -maxdepth 2 -type f | head

```

注意：不建议直接修改这些底层文件。查看和管理数据应优先使用 ChromaDB API 或 Python 客户端。

9.9 清空 ChromaDB 数据

谨慎操作。停止服务并删除 volume：

```
1 docker compose down
2 docker volume rm echomind_chromadb-data
3 docker compose up -d --build
```

如果只想删除某个 collection，可以用 Python 客户端：

代码块

```
1 docker exec -it echomind-app bash
```

代码块

```
1 python - <<'PY'
2 import chromadb
3
4 client = chromadb.HttpClient(host="chromadb", port=8000)
5 client.delete_collection("knowledge_base")
6 print("deleted knowledge_base")
7 PY
```

删除后重启应用，`KnowledgeBase` 会在 collection 为空时重新导入默认文档。

10. Redis 工作记忆查看

Redis 容器名：

代码块

```
1 echomind-redis
```

进入 Redis：

代码块

```
1 docker exec -it echomind-redis redis-cli -a echomind123
```

查看 key：

代码块

```
1 KEYS *
```

工作记忆 key 格式：

代码块

```
1  wm:{user_id}:{conv_id}
```

会话摘要 key 格式：

代码块

```
1  summary:{user_id}:{conv_id}
```

查看某个会话最近消息：

代码块

```
1  LRange wm:user_001:session_001 0 -1
```

查看 TTL：

代码块

```
1  TTL wm:user_001:session_001
```

默认 TTL 是 24 小时。

11. 查看工作记忆压缩内容

工作记忆压缩发生在 `memory/conversation_memory.py` 中。默认配置：

代码块

```
1  WORKING_MAX = 20
2  COMPRESS_AT = 15
```

当同一个 `user_id + conv_id` 的工作记忆达到 15 条消息时，系统会：

代码块

```
1  旧消息 -> LLM 摘要 -> Redis summary
2  旧消息摘要 -> ChromaDB episodic
```


3 最近 5 条消息 -> 继续保留在 Redis wm 列表

日志示例：

代码块

```
1  工作记忆压缩完成：cli_user/5a076f2b-b607-4339-9e9f-f0399862d366，摘要 19 字
```

其中：

代码块

```
1  user_id = cli_user
2  conv_id = 5a076f2b-b607-4339-9e9f-f0399862d366
```

11.1 查看 Redis 中的会话摘要

进入 Redis：

代码块

```
1  docker exec -it echomind-redis redis-cli -a echomind123
```

查询摘要：

代码块

```
1  GET summary:cli_user:5a076f2b-b607-4339-9e9f-f0399862d366
```

一条命令快速查看：

代码块

```
1  docker exec -it echomind-redis redis-cli -a echomind123 \
2  GET summary:cli_user:5a076f2b-b607-4339-9e9f-f0399862d366
```

11.2 查看压缩后仍保留的最近 5 条工作记忆

进入 Redis 后执行：

代码块

```
1 LRange wm:cli_user:5a076f2b-b607-4339-9e9f-f0399862d366 0 -1
```

一条命令快速查看：

代码块

```
1 docker exec -it echomind-redis redis-cli -a echomind123 \  
2 LRange wm:cli_user:5a076f2b-b607-4339-9e9f-f0399862d366 0 -1
```

说明：

- Redis 使用 `LPUSH` 写入，最新消息在列表前面。
- 代码读取时会 `reversed(rows)` 还原时间顺序。
- 压缩后 Redis 工作记忆列表只保留最近 5 条；更早的内容会以摘要形式进入 Redis summary 和 ChromaDB `episodic`。

11.3 查看 ChromaDB 中的情景记忆摘要

如果是全栈部署，应用容器名通常是：

代码块

```
1 echomind-app
```

进入应用容器：

代码块

```
1 docker exec -it echomind-app bash
```

如果你是用 `docker run --rm` 跑 CLI，容器名可能是随机的。先查看：

代码块

```
1 docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Networks}}\t{{.Status}}'
```

进入对应容器：

代码块

```
1 docker exec -it <容器名> bash
```

执行 Python 脚本查询 `episodic` :

代码块

```
1  python - <<'PY'
2  import chromadb
3
4  user_id = "cli_user"
5  conv_id = "5a076f2b-b607-4339-9e9f-f0399862d366"
6
7  client = chromadb.HttpClient(host="chromadb", port=8000)
8  col = client.get_collection("episodic")
9
10 data = col.get(
11     where={"user_id": user_id},
12     include=["documents", "metadatas"],
13 )
14
15 for i, doc in enumerate(data["documents"]):
16     meta = data["metadatas"][i]
17     if meta.get("conv_id") == conv_id:
18         print("=" * 80)
19         print("metadata:", meta)
20         print("summary:", doc)
21         print("full_text_preview:", meta.get("full_text"))
22 PY
```

字段说明:

11.4 如果只想看某个用户的所有情景记忆

代码块

```
1  docker exec -it echomind-app bash
```

```

1 代码块 python - <<'PY'
2  import chromadb
3
4  user_id = "cli_user"
5
6  client = chromadb.HttpClient(host="chromadb", port=8000)
7  col = client.get_collection("episodic")
8
9  data = col.get(
10     where={"user_id": user_id},
11     include=["documents", "metadatas"],
12 )
13
14 for i, doc in enumerate(data["documents"]):
15     print("=" * 80)
16     print("metadata:", data["metadatas"][i])
17     print("summary:", doc)
18 PY

```

11.5 Redis summary 和 ChromaDB episodic 的区别

12. Monitor 在线监控

查看监控摘要：

代码块

```
1 curl http://localhost:8000/monitor
```

响应包含：

代码块

```

1  {
2    "agent_stats": {
3      "general_0": {
4        "total": 10,
5        "success_rate": 1.0,
6        "avg_ms": 1200.3,

```

```
7      "monitor_penalty": 0.0,
8      "routing_score": 0.836
9  }
10 },
11 "tool_stats": {
12     "knowledge_search": {
13         "total": 5,
14         "success_rate": 1.0,
15         "avg_latency_ms": 80.2,
16         "consecutive_fails": 0,
17         "circuit_state": "closed"
18     }
19 },
20 "active_alerts": [],
21 "suggestions": []
22 }
```

指标含义：

Prometheus 页面：

代码块

```
1 http://localhost:9090
```

13. 运行端到端评测

代码块

```
1 curl -X POST http://localhost:8000/eval/run
```

评测内容：

1. 意图识别准确率和 Macro-F1
2. 调用 Orchestrator 生成真实回复
3. LLM-as-Judge 从相关性、准确性、完整性、有用性打分
4. 与上一次评测结果做回归检测
5. 生成优化建议

响应示例：

代码块

```
1  {
2    "pass_rate": 0.83,
3    "total": 5,
4    "passed": 4,
5    "avg_scores": {
6      "intent_accuracy": 0.875,
7      "relevance": 0.88,
8      "accuracy": 0.82,
9      "completeness": 0.79,
10     "helpfulness": 0.85
11   },
12   "regressions": [],
13   "recommendations": [
14     "意图识别准确率 < 90%: 增加 Few-shot 示例, 或对低 F1 的意图类别补充训练数据"
15   ],
16   "results": []
17 }
```

14. 停止、重启和清理

停止服务：

代码块

```
1  docker compose stop
```

重启服务：

代码块

```
1  docker compose restart echomind
```

停止并删除容器，但保留数据卷：

代码块

```
1 docker compose down
```

停止并删除容器和数据卷：

代码块

```
1 docker compose down -v
```

重新构建并启动：

代码块

```
1 docker compose up -d --build
```

15. 常见问题

15.1 `/health` 返回 503

查看应用日志：

代码块

```
1 docker compose logs -f echomind
```

重点检查：

- `.env` 是否配置 `ANTHROPIC_API_KEY`
- Redis 是否健康
- ChromaDB 是否健康
- 应用容器是否正在反复重启

15.2 ChromaDB 连接失败

查看 ChromaDB 状态：

代码块

```
1 docker compose ps chromadb
```

```
2 docker compose logs -f chromadb
3 curl http://localhost:8001/api/v1/heartbeat
```

应用容器内测试：

代码块

```
1 docker exec -it echomind-app bash
2 python - <<'PY'
3 import chromadb
4 client = chromadb.HttpClient(host="chromadb", port=8000)
5 print(client.heartbeat())
6 PY
```

15.3 Redis 认证失败

确认 `.env` 和 `docker-compose.yml` 中使用的密码一致。默认密码是：

代码块

```
1 echomind123
```

测试连接：

代码块

```
1 docker exec -it echomind-redis redis-cli -a echomind123 ping
```

15.4 /search 没有结果

先确认识别库中有数据：

代码块

```
1 curl http://localhost:8000/knowledge/stats
```

如果是 0，可以重新导入演示文档：

代码块

```
1 curl -X POST http://localhost:8000/knowledge/upload \
2 -F "file=@data/demo_docs/sample_knowledge.json"
```


再测试：

代码块

```
1 curl -X POST "http://localhost:8000/search?query=API如何接入&top_k=3"
```

15.5 用户画像查不到

用户画像是异步更新的，并且依赖 LLM 调用成功。排查步骤：

1. 先调用 `/chat`，使用固定 `user_id`
2. 等待几秒
3. 查看 `docker compose logs -f echomind` 是否出现 `用户画像已更新`
4. 使用第 8.6 节的 Python 脚本查询 `user_profile`

15.6 情景记忆查不到

情景记忆不是每次对话都写入。只有当前会话消息数达到压缩阈值后才写入。默认阈值：

代码块

```
1 MemoryManager.COMPRESS_AT = 15
```

连续发 16 条以上消息后再查看 `episodic`。

16. 推荐验证流程

完整验证可以按这个顺序执行：

代码块

```
1 # 1. 启动
2 docker compose up -d --build
3
4 # 2. 健康检查
5 curl http://localhost:8000/health
6
7 # 3. 主对话
8 curl -X POST http://localhost:8000/chat \
9     -H "Content-Type: application/json" \
10     -d '{"message": "你好，我想了解退款政策", "user_id": "demo_user", "conv_id": "demo_conv"}'
11
12 # 4. 知识库统计
13 curl http://localhost:8000/knowledge/stats
```

```
14
15 # 5. 导入演示知识库
16 curl -X POST http://localhost:8000/knowledge/upload \
17     -F "file=@data/demo_docs/sample_knowledge.json"
18
19 # 6. 检索
20 curl -X POST "http://localhost:8000/search?query=EchoMind如何接入API&top_k=3"
21
22 # 7. 监控
23 curl http://localhost:8000/monitor
24
25 # 8. Skills
26 curl http://localhost:8000/skills
27
28 # 9. 评测
29 curl -X POST http://localhost:8000/eval/run
```